# Branching and boundary tests

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

---

## Before you start

Does minutes <= 12 accept exactly 12? How does < differ?

Plan time to read, investigate and explain. Keep predictions separate from observations.

---



## Your mission

> Start with the target: Trace the selected branch in an if elif else chain. Try the recall before opening notes.

### Your learning target

I can trace the selected **branch** in an if elif else chain.

I can use **indentation** to identify which statements belong to a **branch**.

I can test values at and around a decision **boundary**.

### Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Boolean **condition**s. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

This week you will practise:

Trace the selected **branch** in an if elif else chain

Use **indentation** to identify which statements belong to a **branch**

Test values at and around a decision **boundary**

### Retrieve what you already know

> Does minutes <= 12 accept exactly 12? How does < differ?

> **My starting explanation**

### This week's end goal

Trace **branch**ing and **boundary** tests and explain the changed program before running it.

---

STUDENT LESSON · 2 OF 18

## Notice the evidence

> Investigate the supplied Branching and boundary tests case. Separate observations, given facts and assumptions before choosing an explanation.

A fictional activity sorter labels a supplied integer duration. Negative values are Invalid; values from zero through 12 are Short; larger values are Long. The labels describe duration only.

Write this requirement before the program so that a test has an expected answer independent of the code.

An if statement selects a block when its **condition** is true. An elif is considered only if earlier **condition**s in the same chain were false. An else handles the remaining cases. Colons end the **condition** lines.

Use four spaces for the indented statements and keep related **branch**es aligned.

> **List the given facts and one fact that is not supplied.**
>
> _____
>
> _____
>
> _____
>
> _____

---

STUDENT LESSON · 3 OF 18

## See how it works

> Follow the mechanism for Branching and boundary tests. Connect each labelled input, deciding step and result.

### One chain chooses one label

| INPUT | CHECKS IN ORDER | SELECTED LABEL |
| --- | --- | --- |
| -1 | Below zero is True | Invalid |
| 12 | Below zero False; <= 12 True | Short |
| 13 | Both checks False | Long |

Later branches in this chain are skipped after a condition succeeds.

```
minutes = 12
if minutes < 0:
    label = "Invalid"
elif minutes <= 12:
    label = "Short"
else:
    label = "Long"
print(label)
```

The **output** is Short. For 12, the first **condition** is false and the second is true. The else block is skipped. print(label) is not indented, so it runs after the selected **branch**.

Each path assigns label before this line reads it. Change only minutes and save before each fresh run.

### Connect the picture and the explanation

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

# Compare the cases

> Compare the supplied cases for Branching and boundary tests. Identify the detail that changes the conclusion.

### Compare a familiar case with a changed case

For -1, 12 and 13, record checked **condition**s, skipped **condition**s and the final label. Run each as a fresh script after predicting.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

### Change one thing in the pictured case

#### Change one thing

Change: Remove the first negative-count **branch**.

#### Predicted result

**Prediction**: A count of -2 would incorrectly receive small.

#### Why the result follows

Reason: Negative two is less than five, but it is not a permitted measured count.

STUDENT LESSON · 5 OF 18

## Words that unlock the idea

> Use the vocabulary of Branching and boundary tests to explain a concrete decision, rather than only memorising definitions.

| Word | Meaning |
| --- | --- |
| Branch | A possible path through a program |
| Condition | An expression used to decide which path to take |
| Indentation | Leading spaces that identify a Python code block |
| Boundary | A value where a classification may change |
| Regression check | A previous test rerun after an edit to detect a new failure |

**Use two terms to explain a step in this week's example.**

STUDENT LESSON · 6 OF 18

## Follow a worked example

> Follow the worked trace for Branching and boundary tests, predict a step, then compare with its explanation.

| Input minutes | Expected label | Reason for this test |
| --- | --- | --- |
| -1 | Invalid | Negative input rejection |
| 0 | Short | Lowest valid duration |
| 11 | Short | Just below the upper boundary |
| 12 | Short | Exact upper boundary |
| 13 | Long | Just above the boundary |

These tests exercise every **branch** and the equality case. A change from <= 12 to < 12 is exposed by 12; testing 11 alone misses it.

**Branch** coverage means every **branch** was exercised, but it does not prove every **condition** is correct.

> **Suppose the program uses < 12 instead of <= 12. What is printed for 12, why does that violate the written rule, and which earlier test should still pass after repair?**

### Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

### A pictorial worked case: Branching and boundary tests

Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

### The facts we will use

**Supplied facts**

Python 3: if count<0: label="invalid"; elif count<5: label="small"; else: label="large". Read as indented **branch**es.

AI ACADEMY / PYTHON-BRIDGE/5

## Branching and boundary tests

**Start with the supplied facts**

Follow each arrow. Ask which fact allows the next step.



**1. Check validity**

For count=-2, the first condition selects invalid.





**2. Check small boundary**

For count=4, validity passes and count<5 selects small.





**3. Handle equality**

For count=5, both earlier conditions fail, so large is selected.

**CHANGE ONE THING**

Remove the first negative-count branch.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

### Read the picture one step at a time

| STEP | WHAT HAPPENS AND WHY |
| --- | --- |
| 1. Check validity | For count=-2, the first condition selects invalid. |
| 2. Check small boundary | For count=4, validity passes and count<5 selects small. |
| 3. Handle equality | For count=5, both earlier conditions fail, so large is selected. |

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

### Why this result follows

#### Why the result follows

An if/elif/else chain selects the first matching **branch**. Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

### What this example does not establish

#### Keep the stated limit

The toy procedure assumes numeric counts; validate types and define missing-value behaviour before using it in a real service.

### Keep the stated limit

Read the labelled picture: Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

### Keep the stated limit

Change one thing: Remove the first negative-count **branch**. Predict the result before checking the explanation. What remains the same?

---

STUDENT LESSON · 7 OF 18

## Find and repair a mistake

> Find the unsupported step in the Branching and boundary tests claim and explain a repair using the supplied evidence.

```
minutes = -1
if minutes < 0:
    label = "Invalid"
if minutes <= 12:
    label = "Short"
else:
    label = "Long"
print(label)
```

### Check the claim against the facts

This faulty version prints Short. The second if starts a separate decision and overwrites Invalid. Its else belongs to that second if. Restoring elif makes the second test depend on the first test failing.

> Moving print inside one **branch** can also hide **output** on other paths; **indentation** affects behavior, not just appearance.

The script rejects negative integers, but it does not validate arbitrary types or check a measured duration. A supplied text value can raise an error during comparison.

Keep the stated integer **input** contract and report the limitation honestly.

### Locate the earliest unsupported step

Trace the two-independent-if example for -1. Identify which assignment overwrites the earlier label and make the smallest repair.

**State one repair and the checks you would rerun.**

---

STUDENT LESSON · 8 OF 18

## Practise together

> Practise Branching and boundary tests with support, then explain the deciding step in your own words.

Describe the purpose of if, elif and else in the worked duration sorter. Explain why print(label) is outside the indented **branch**es.

**Suppose the program uses < 12 instead of <= 12. What is printed for 12, why does that violate the written rule, and which earlier test should still pass after repair?**

**Explain your prediction to a partner before comparing results.**

### Practise with less help: Trace the tested path

Use workbook W2 for the connected example "**Branch**ing and **boundary** tests". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

| Plan | Do | Check |
| --- | --- | --- |
| What is given? Which rule or procedure applies? | Record each intermediate step. | Does the result follow? What remains unknown? |

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

### Check your reasoning

Use the worked case for **Branch**ing and **boundary** tests. Proposed explanation: "Every **condition** in the chain always runs". Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

### Explain the picture

Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

**My explanation using a pictured step and the given facts**

This is supported practice. Keep the separately named independent case for an attempt before feedback.

---

STUDENT LESSON · 9 OF 18

# Run the investigation

> Investigate Branching and boundary tests. Record the prediction, actual observation or labelled paper trace, explanation and next test separately.

### Predict, investigate and preserve the evidence

For -1, 12 and 13, record checked **condition**s, skipped **condition**s and the final label. Run each as a fresh script after predicting.

```
minutes = 12
if minutes < 0:
    label = "Invalid"
elif minutes <= 12:
    label = "Short"
else:
    label = "Long"
print(label)
```

```
minutes = -1
if minutes < 0:
    label = "Invalid"
if minutes <= 12:
    label = "Short"
else:
```

```
    label = "Long"
print(label)
```

| Record | Your evidence |
| --- | --- |
| Given input and fixed rule | |
| Prediction before checking | |
| Actual result, or “not run” | |
| First mismatch and one explanation | |
| Boundary or missing input and expected response | |

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

**Plan → try → check**

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

---

STUDENT LESSON · 10 OF 18

## Build your understanding

> Move from recall to tracing, diagnosis and transfer in Branching and boundary tests. Explain why each step is justified.

**Explain → apply → challenge**

Describe the purpose of if, elif and else in the worked duration sorter. Explain why print(label) is outside the indented **branch**es.

Trace the two-independent-if example for -1. Identify which assignment overwrites the earlier label and make the smallest repair.

A repair changes the small-count **condition** from < 6 to <= 6. Name the case that exposes the original error and two different kinds of earlier cases to rerun.

> **Explain which evidence supports your conclusion and which claim remains unproved.**

---

STUDENT LESSON · 11 OF 18

## Connect your project

> Apply this week's idea to Python readiness portfolio. Keep an annotated decision and the evidence supporting it.

Add a labelled **branch**ing and **boundary** tests **evidence** sheet to Python readiness portfolio. Show how you can trace the selected **branch** in an if elif else chain.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

| Project evidence | Record |
| --- | --- |
| Component and version | |
| Inputs permitted at prediction time | |
| Trace and comparison with earlier work | |
| Failure, limit and human review | |

### Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

---

STUDENT LESSON · 12 OF 18

## Show what you know

> Use a genuinely new case for independent evidence. Previously discussed examples remain practice; use the separately named independent case before feedback.

For each run, record **input**, expected label, **condition**s actually checked and observed label. Mark skipped **condition**s as skipped rather than pretending the program evaluated them.

After a repair, rerun earlier cases as well as the failing one. Week 6 will repeat a block and track changes across iterations.

> **A learner tests -1, 2 and 20 and concludes the sorter is completely correct because all three branches ran. Explain what important case is missing and why branch coverage alone is insufficient.**

A repair changes the small-count **condition** from < 6 to <= 6. Name the case that exposes the original error and two different kinds of earlier cases to rerun.

**My limited claim and a useful next check**

STUDENT LESSON · 13 OF 18

# Study a complete case

> Revisit the complete Branching and boundary tests case for supported practice. Repeated examples do not become fresh assessments.

### Complete evidence record

#### One chain chooses one label

| INPUT | CHECKS IN ORDER | SELECTED LABEL |
| --- | --- | --- |
| -1 | Below zero is True | Invalid |
| 12 | Below zero False; <= 12 True | Short |
| 13 | Both checks False | Long |

Later branches in this chain are skipped after a condition succeeds.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

> **My complete trace of the worked case**
>
> | | |
> |---|---|
> | | |
> | | |
> | | |

---

STUDENT LESSON · 14 OF 18

## Try a new case independently

> Attempt the reserved new case before feedback. Record support; the earlier examples below remain additional practice.

### Independent case: Trace branch order

Predict the action for 30, then trace 29 and -1. Explain why validity is checked before the lamp decision.

```
reading = 30
if not 0 <= reading <= 100:
    action = "invalid"
elif reading < 30:
    action = "on"
else:
    action = "off"
print(action)
```

**My independent explanation, trace and limits**

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

### Attempt this without hints

Use an integer tiles count. Label a negative value Invalid, zero Empty, 1 through 6 Small and values above 6 Large. Write an ordered if elif else script and predict -1, 0, 1, 6 and 7. Include one print after the chain.

**My independent procedure, result and limitation**

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

### Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

## Your lesson route

> Use this map to navigate Branching and boundary tests, connect the goals to tasks and keep one evidence record for each attempt.

Start with this question: Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

| By the end, I can | How I will show it |
| --- | --- |
| Trace the selected branch in an if elif else chain | Explain the deciding step, show a trace or test, and state one limit. |
| Use indentation to identify which statements belong to a branch | Explain the deciding step, show a trace or test, and state one limit. |
| Test values at and around a decision boundary | Explain the deciding step, show a trace or test, and state one limit. |

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

---

STUDENT LESSON · 16 OF 18

## Finish and return

> Explain Branching and boundary tests without notes, identify a remaining question and plan a later check with a changed case.

Close your notes. Explain this goal in your own words: Trace the selected **branch** in an if elif else chain

**My deciding step, evidence and one remaining question**

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

---

STUDENT LESSON · 17 OF 18

## Supported practice, repair and portfolio

> Attempt supported questions before using hints or feedback. Review a missing building block, then keep the completed work.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

### Practice 1: explain the deciding evidence

Which labelled step matches this detail? For count=5, both earlier **condition**s fail, so large is selected.

1. Check validity

2. Handle equality

3. Check small **boundary**

**My selected conclusion, deciding fact and explanation**

### Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

### Feedback · compare after trying

After your attempt, compare with the supported explanation: Handle equality For count=5, both earlier **condition**s fail, so large is selected. An if/elif/else chain selects the first matching **branch**.

Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

**Practice 2: explain the deciding evidence**

### Changed case · predict first

Changed-case practice: Remove the first negative-count **branch**. Which conclusion is supported by the supplied facts?

1. This one example guarantees correct results for every future **input**.

2. A count of -2 would incorrectly receive small.

3. We can decide the result without examining the changed **input** or the procedure.

**My selected conclusion, deciding fact and explanation**

### Hint · try first

Hint if needed: Use the changed **condition** and the deciding step in the pictorial trace. The answer must say what follows from those facts.

### Feedback · compare after trying

After your attempt, compare with the supported explanation: A count of -2 would incorrectly receive small. Negative two is less than five, but it is not a permitted measured count.

**Repair a missing building block**

Review Boolean **condition**s (python-bridge/4). Goal: Translate stated requirements into comparisons. Short practice: Give both comparison results for ages 11, 15 and 16.

Which comparison rejects each outside value, and why does and matter?. Return to this week and explain what you changed.

Review Values variables and types (python-bridge/2). Goal: Distinguish integer, floating-point, text and Boolean values. Short practice: Predict both print **output**s and explain why they differ.

Does count+tag work, and what happens to each valid **output** after count becomes ten?. Return to this week and explain what you changed.

### Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

### Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

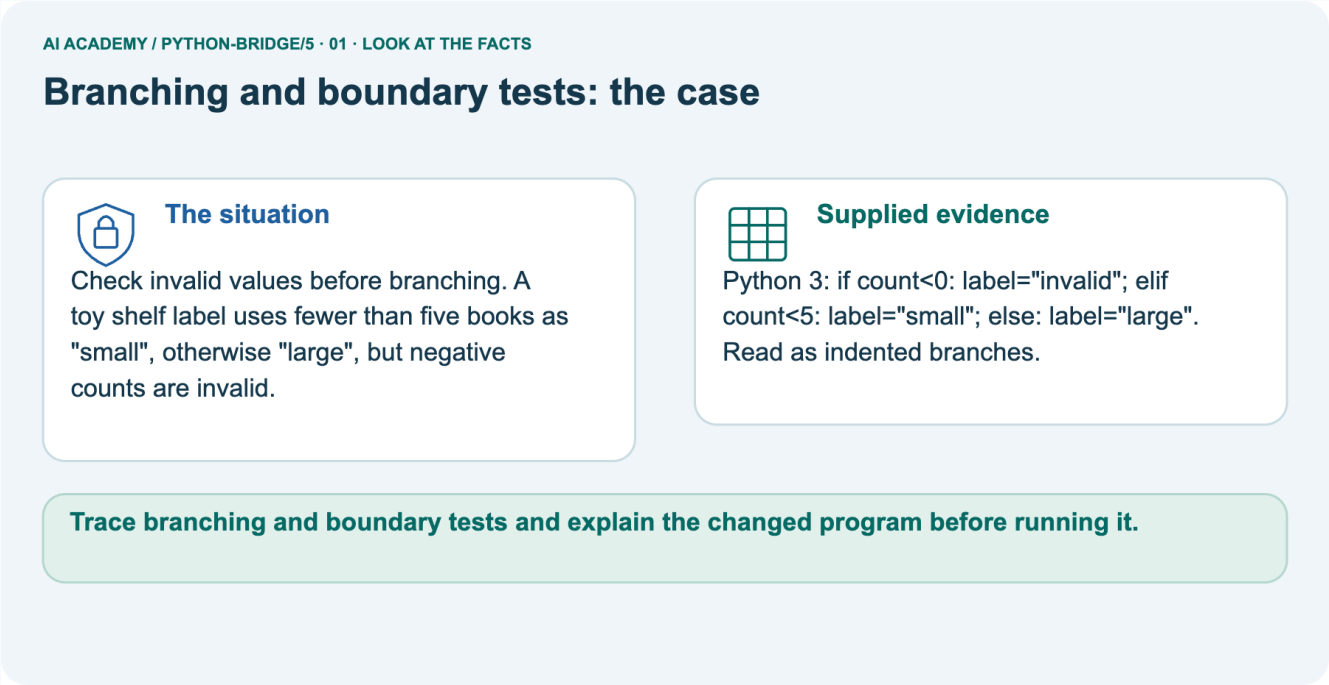
---

STUDENT LESSON · 18 OF 18

## Visual concept journey

> Explain each Branching and boundary tests graphic using its labels, deciding evidence and relevant limitation.

## 01. Branching and boundary tests: the case



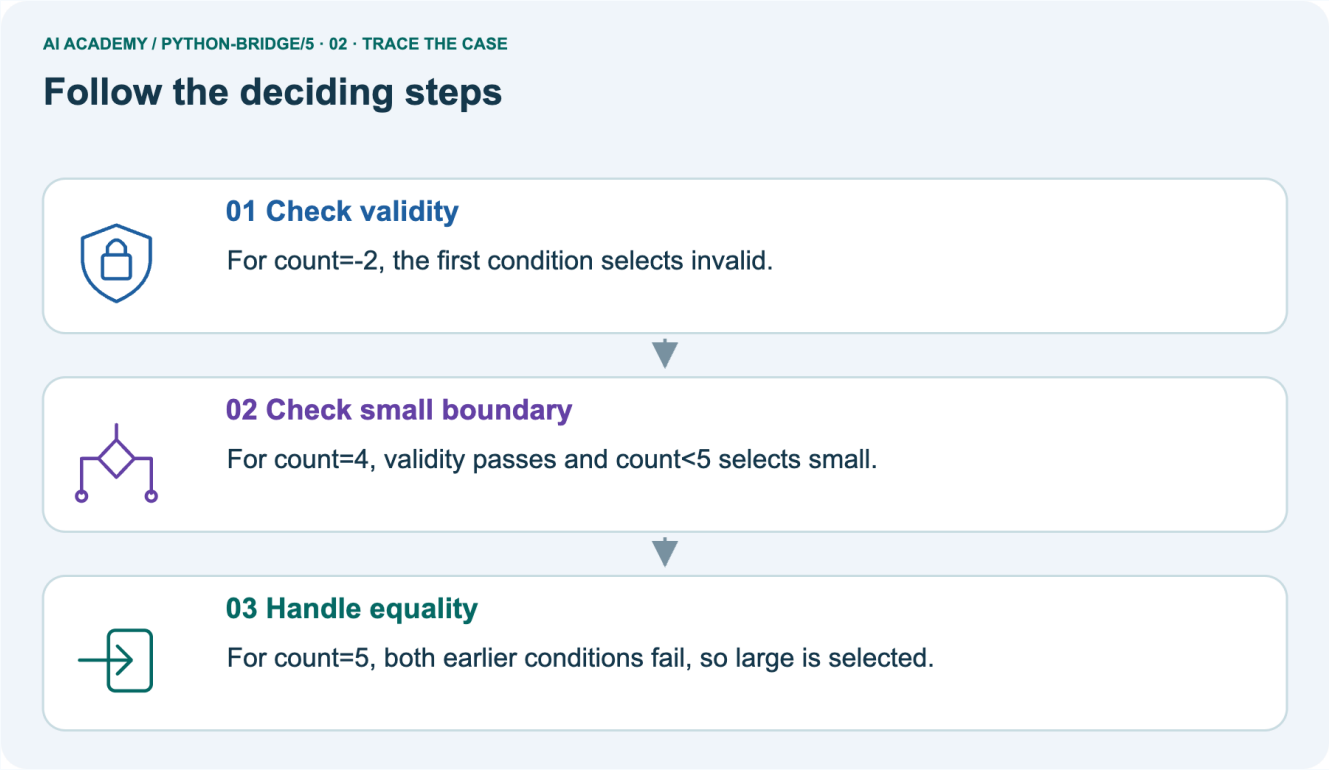

Follow the labelled picture with the matching explanation. Open the vector diagram

Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

Supplied facts: Python 3: if count<0: label="invalid"; elif count<5: label="small"; else: label="large". Read as indented **branch**es.

> Point to the given fact that matters most. Explain why it matters.

## 02. Follow the deciding steps

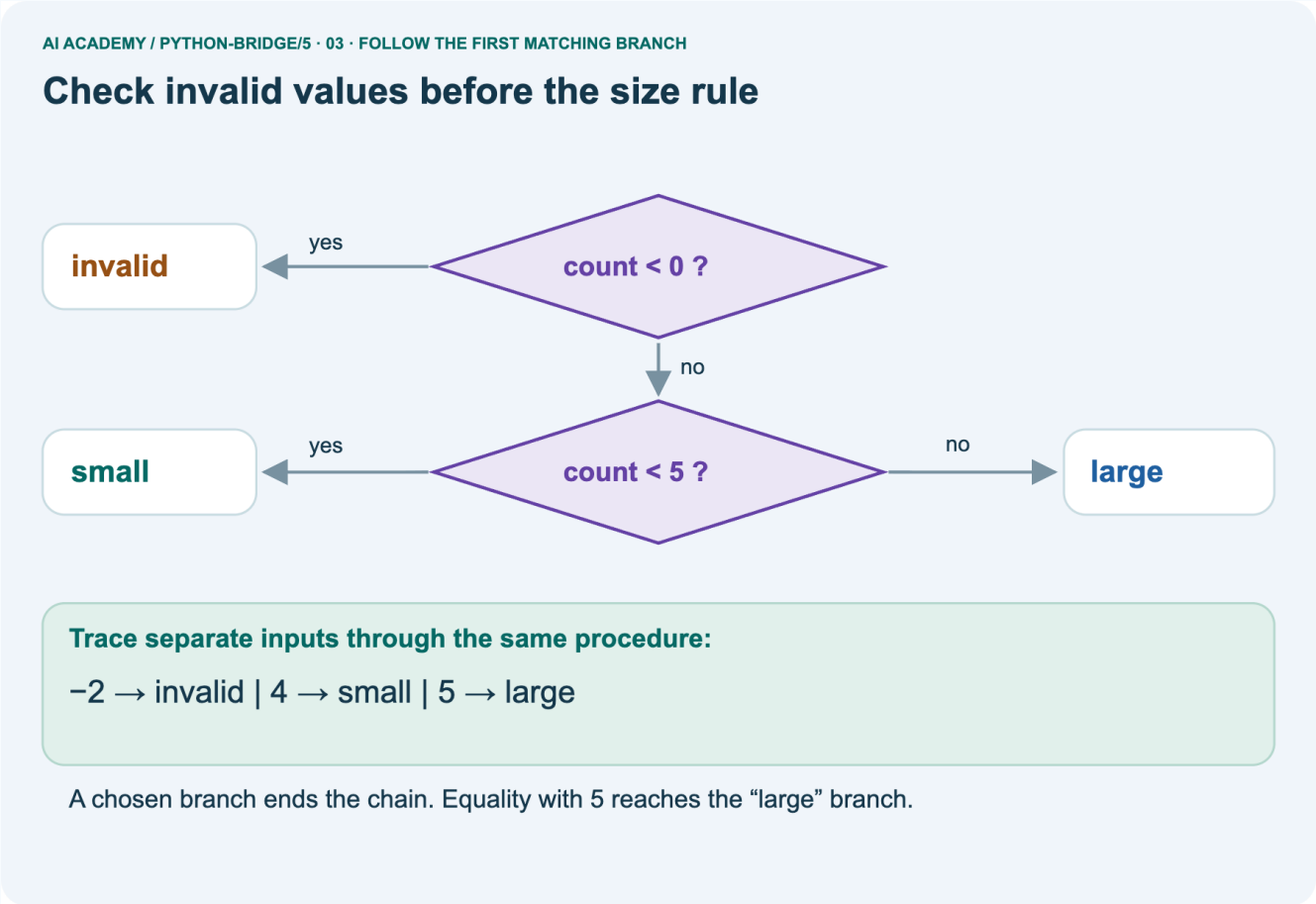



Follow the labelled picture with the matching explanation. Open the vector diagram

Check validity: For count=-2, the first **condition** selects invalid. → Check small **boundary**: For count=4, validity passes and count<5 selects small. → Handle equality: For count=5, both earlier **condition**s fail, so large is selected.

> Point to each numbered stage and explain which recorded **input** it uses.

### 03. Check invalid values before the size rule

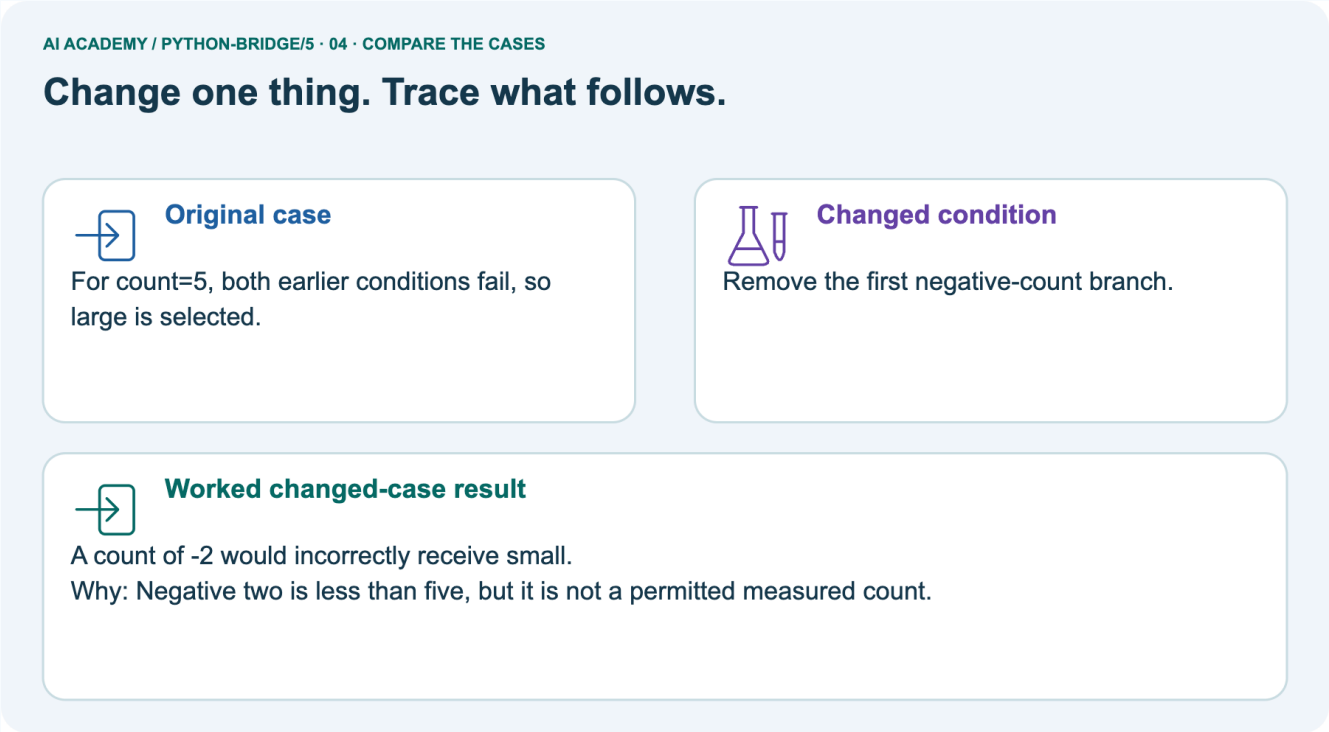



Follow the labelled picture with the matching explanation. Open the vector diagram

First ask count less than zero: yes selects invalid. Otherwise ask count less than five: yes selects small; no selects large. Separate **input** traces are minus two invalid, four small, five large.

> Read the labels and values. Explain the relationship the model shows.

## 04. Change one thing. Trace what follows.



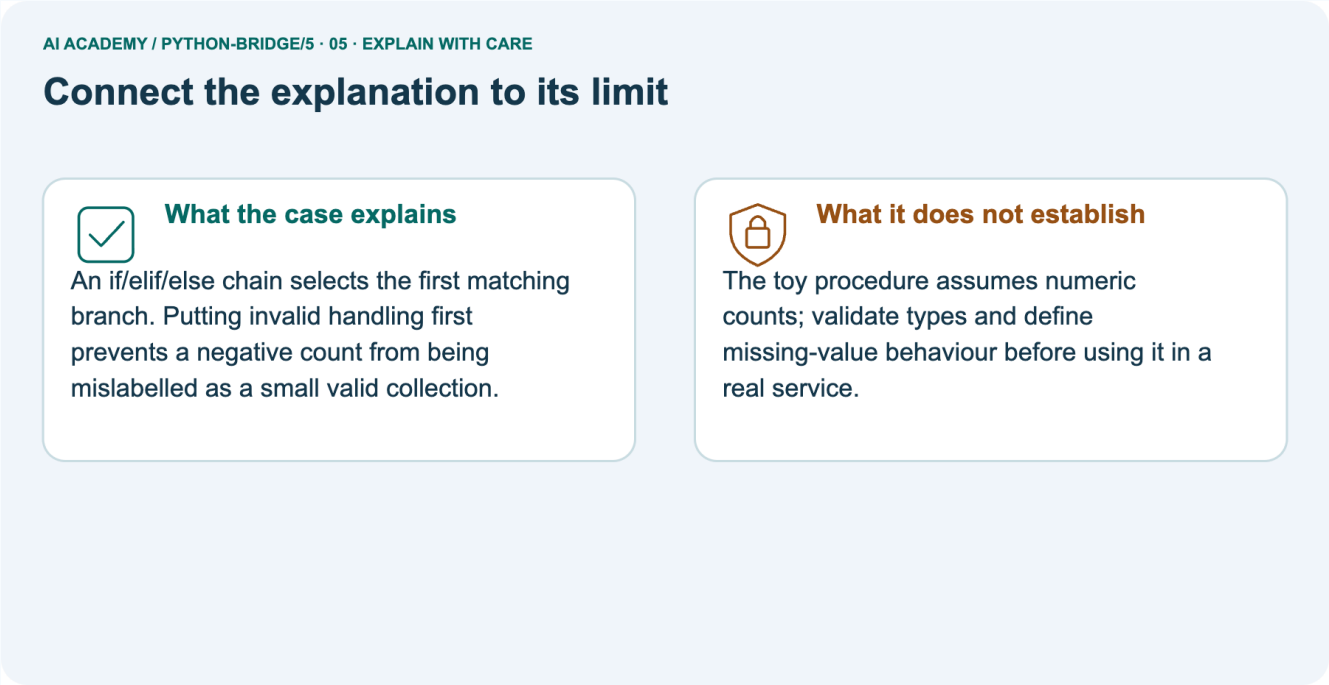

Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: For count=5, both earlier **condition**s fail, so large is selected. Changed **condition**: Remove the first negative-count **branch**. Expected result: A count of -2 would incorrectly receive small.

Reason: Negative two is less than five, but it is not a permitted measured count.

> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

## 05. Connect the explanation to its limit



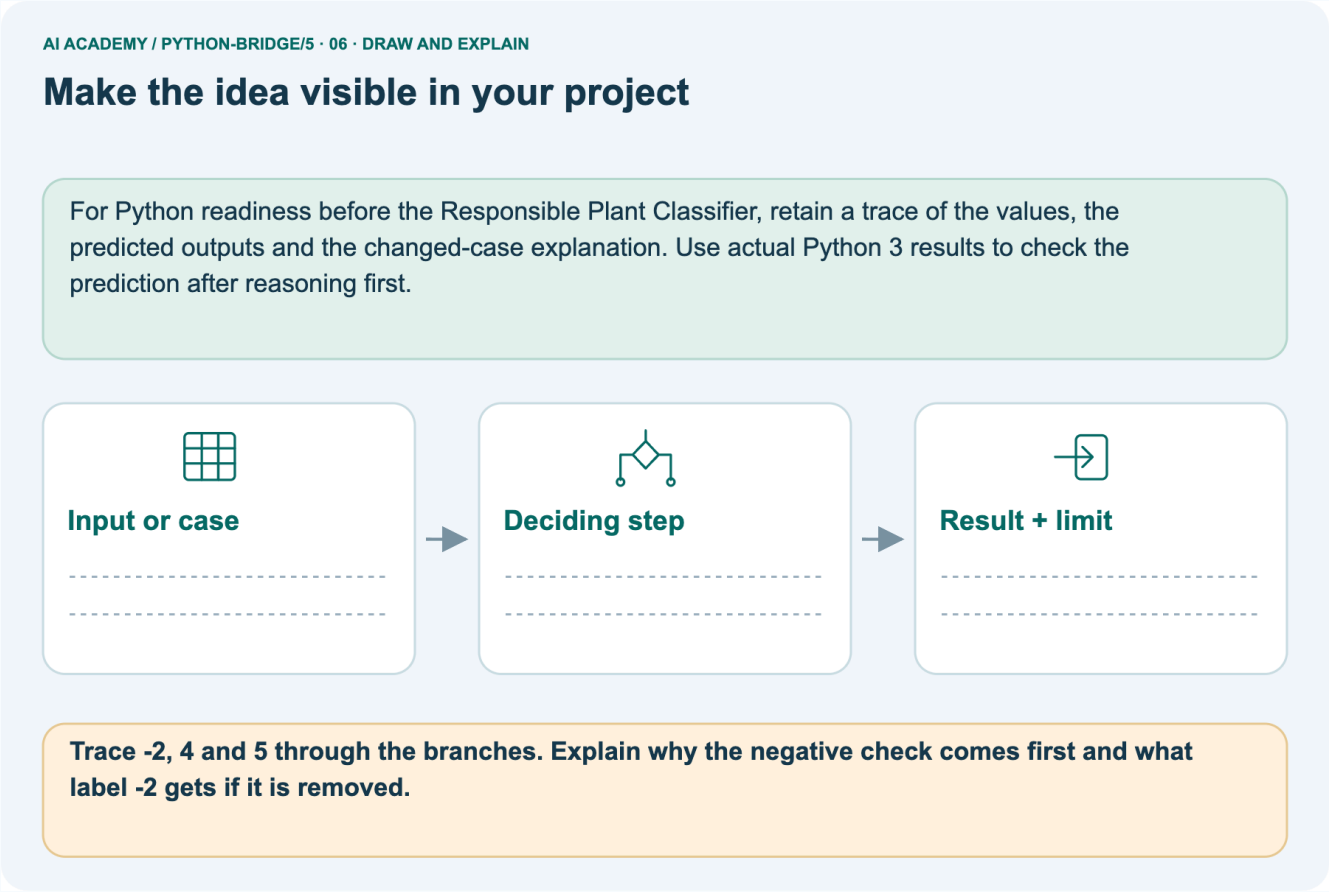

Follow the labelled picture with the matching explanation. Open the vector diagram

An if/elif/else chain selects the first matching **branch**. Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

Limit: The toy procedure assumes numeric counts; validate types and define missing-value behaviour before using it in a real service.

> Explain the case, then point to a limit on the claim.

## 06. Make the idea visible in your project




Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

> Draw your project trace and justify the deciding step using **evidence**.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.